

Introduction into Machine Learning and Big Data

Tutorial 3 – Decision Trees, Random Forests

Task 0: Installing the necessary requirements

Please install all the necessary requirements via:

```
pip install numpy pandas matplotlib scikit-learn xgboost
```

Task 1: Visualize the Data Set

Visualize the dataset. Use different colors for pass and fail:

```
plt.figure(figsize=(8, 6))

plt.scatter(
    df[df["pass"] == 0]["hours_studied"],
    df[df["pass"] == 0]["attendance"],
    label="Fail",
    alpha=0.7
)

plt.scatter(
    df[df["pass"] == 1]["hours_studied"],
    df[df["pass"] == 1]["attendance"],
    label="Pass",
    alpha=0.7
)

plt.xlabel("Hours Studied")
plt.ylabel("Attendance (%)")
plt.title("Student Pass/Fail Dataset")
plt.legend()
plt.show()
```

Task 2: Training A Decision Tree

Train a DecisionTreeClassifier using the dataset. Experiment with:

```
max_depth = 1
max_depth = 3
max_depth = None
```

For each model:

1. Train the model
2. Evaluate:
 - Training accuracy
 - Test accuracy
3. Output:
 - Tree depth
 - Number of leaves

Code for creating a Decision Tree:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X = df[["hours_studied", "attendance"]]
y = df["pass"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

depths = [1, 3, None]

for depth in depths:
    model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42
    )
```

```

model.fit(X_train, y_train)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("max_depth:", depth)
print("Train Accuracy:", accuracy_score(y_train, train_pred))
print("Test Accuracy:", accuracy_score(y_test, test_pred))
print("Tree Depth:", model.get_depth())
print("Number of Leaves:", model.get_n_leaves())
print("-" * 40)

```

Task 3: Evaluating the Decisions

Implement a function to visualize decision boundaries, then apply it to all decision tree models from Task 2. How do the decision boundaries change with depth?

Code for creating the plots:

```

def plot_decision_boundary(model, X, y, title):
    h = 0.1

    x_min, x_max = X["hours_studied"].min() - 1,
    X["hours_studied"].max() + 1
    y_min, y_max = X["attendance"].min() - 5, X["attendance"].max() + 5

    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, h),
        np.arange(y_min, y_max, h)
    )

    grid = pd.DataFrame({
        "hours_studied": xx.ravel(),
        "attendance": yy.ravel()
    })

    Z = model.predict(grid)
    Z = Z.reshape(xx.shape)

```

```

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(
    X[y == 0]["hours_studied"],
    X[y == 0]["attendance"],
    label="Fail",
    alpha=0.7
)

plt.scatter(
    X[y == 1]["hours_studied"],
    X[y == 1]["attendance"],
    label="Pass",
    alpha=0.7
)

plt.xlabel("Hours Studied")
plt.ylabel("Attendance (%)")
plt.title(title)
plt.legend()
plt.show()

```

Code for using the model:

```

for depth in [1, 3, None]:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)

    plot_decision_boundary(
        model,
        X_train,
        y_train,
        title=f"Decision Tree, max_depth={depth}"
    )

```

Task 4: Training a Random Forest

Train a RandomForestClassifier. Experiment with:

```
n_estimators = 1
n_estimators = 5
n_estimators = 100
```

For each model:

1. Train the model
2. Evaluate training and test accuracy
3. Plot decision boundaries

Code for training the model:

```
from sklearn.ensemble import RandomForestClassifier

for n in [1, 5, 100]:
    rf = RandomForestClassifier(
        n_estimators=n,
        max_features="sqrt",
        random_state=42
    )

    rf.fit(X_train, y_train)

    train_pred = rf.predict(X_train)
    test_pred = rf.predict(X_test)

    print("n_estimators:", n)
    print("Train Accuracy:", accuracy_score(y_train, train_pred))
    print("Test Accuracy:", accuracy_score(y_test, test_pred))
    print("-" * 40)

    plot_decision_boundary(
        rf,
        X_train,
        y_train,
        title=f"Random Forest, n_estimators={n}"
    )
```

Task 5: Random Forest Hyperparameters

Train a RandomForestClassifier.

Investigate the effect of the following parameters:

- max_depth
- max_features
- min_samples_leaf

Test at least the following configurations:

- (1) max_depth=1
- (2) max_depth=3
- (3) max_depth=None
- (4) max_features=1 vs 2 vs "sqrt"
- (5) min_samples_leaf=1 vs 10

Store results in a table including:

- Training accuracy
- Test accuracy

Code for training the model:

```
configs = [  
    {"n_estimators": 100, "max_depth": 1, "max_features": "sqrt",  
     "min_samples_leaf": 1},  
    {"n_estimators": 100, "max_depth": 3, "max_features": "sqrt",  
     "min_samples_leaf": 1},  
    {"n_estimators": 100, "max_depth": None, "max_features": "sqrt",  
     "min_samples_leaf": 1},  
    {"n_estimators": 100, "max_depth": None, "max_features": 2,  
     "min_samples_leaf": 1},  
    {"n_estimators": 100, "max_depth": None, "max_features": "sqrt",  
     "min_samples_leaf": 10},  
]  
  
results = []  
  
for config in configs:  
    rf = RandomForestClassifier(  
        **config,  
        random_state=42
```

```

    )

    rf.fit(X_train, y_train)

    train_acc = accuracy_score(y_train, rf.predict(X_train))
    test_acc = accuracy_score(y_test, rf.predict(X_test))

    results.append({
        **config,
        "train_accuracy": train_acc,
        "test_accuracy": test_acc
    })

rf_results = pd.DataFrame(results)
rf_results

```

Task 6: Extra-Trees

Train an ExtraTreesClassifier. Experiment with:

```

n_estimators = 1, 5, 100
max_depth = 1, 3, None
max_features = 1, 2, "sqrt"

```

For each configuration:

1. Train the model
2. Measure training and test accuracy
3. Plot decision boundaries

Code for training the model:

```

from sklearn.ensemble import ExtraTreesClassifier

extra_configs = [
    {"n_estimators": 1, "max_depth": None, "max_features": "sqrt"},
    {"n_estimators": 5, "max_depth": None, "max_features": "sqrt"},
    {"n_estimators": 100, "max_depth": None, "max_features": "sqrt"},
    {"n_estimators": 100, "max_depth": 1, "max_features": "sqrt"},
    {"n_estimators": 100, "max_depth": 3, "max_features": "sqrt"},

```

```

{"n_estimators": 100, "max_depth": None, "max_features": 1},
{"n_estimators": 100, "max_depth": None, "max_features": 2},
]

```

```
extra_results = []
```

```
for config in extra_configs:
```

```

    et = ExtraTreesClassifier(
        **config,
        random_state=42
    )

```

```
et.fit(X_train, y_train)
```

```
train_acc = accuracy_score(y_train, et.predict(X_train))
```

```
test_acc = accuracy_score(y_test, et.predict(X_test))
```

```

extra_results.append({
    **config,
    "train_accuracy": train_acc,
    "test_accuracy": test_acc
})

```

```

plot_decision_boundary(
    et,
    X_train,
    y_train,
    title=f"Extra-Trees: {config}"
)

```

```
extra_results = pd.DataFrame(extra_results)
```

```
extra_results
```

Task 6: Extra-Trees

Train one final version of each model:

- Decision Tree

- Random Forest
- Extra-Trees

Create a comparison table including:

- Training accuracy
- Test accuracy
- Overfitting gap (train - test)
- Key hyperparameters

Code:

```
models = {
    "Decision Tree": DecisionTreeClassifier(
        max_depth=3,
        random_state=42
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        max_depth=None,
        max_features="sqrt",
        random_state=42
    ),
    "Extra-Trees": ExtraTreesClassifier(
        n_estimators=100,
        max_depth=None,
        max_features="sqrt",
        random_state=42
    )
}

comparison = []

for name, model in models.items():
    model.fit(X_train, y_train)

    train_acc = accuracy_score(y_train, model.predict(X_train))
    test_acc = accuracy_score(y_test, model.predict(X_test))
```

```
comparison.append({  
    "model": name,  
    "train_accuracy": train_acc,  
    "test_accuracy": test_acc,  
    "overfitting_gap": train_acc - test_acc  
})
```

```
comparison_df = pd.DataFrame(comparison)  
comparison_df.sort_values("test_accuracy", ascending=False)
```